Architecture d'un système d'exploitation

TD1 — Interaction de la pagination avec les caches

ASE 2025-2026

Problématique

Ce TD s'intéresse au fonctionnement de la mémoire sur une architecture moderne, et en particulier à la manière dont la pagination — mécanisme par lequel le système d'exploitation traduit les adresses virtuelles manipulées par les programmes en adresses physiques réellement présentes en RAM — interagit avec les caches processeur. On cherche à comprendre pourquoi des données contiguës du point de vue d'un programme ne le sont pas nécessairement en mémoire physique, quel impact cela a sur les performances, et comment on peut s'affranchir de ce problème. Dans un second temps, on met ces notions en pratique en concevant un allocateur mémoire en espace utilisateur capable de garantir la contiguïté physique de ses allocations.

1 Comprendre l'interaction entre pagination et caches

1.1 Le problème des collisions de lignes de cache

Un cache associatif organise ses lignes en un nombre limité de voies : chaque adresse mémoire ne peut être placée que dans un sous-ensemble précis de ces voies. Or, sous Linux, les pages physiques sont attribuées de manière quasiment aléatoire aux applications. Il est donc tout à fait possible qu'une application reçoive plusieurs pages physiques dont l'alignement correspond aux mêmes voies du cache. Si le nombre de telles pages dépasse le nombre de voies disponibles, et que l'application accède régulièrement à l'ensemble de ces données, il se produit des évictions et rechargements permanents des lignes de cache : l'application est ralentie, alors même que la totalité de ses données tiendrait dans le cache si elles étaient correctement réparties. C'est précisément ce phénomène que le TD propose d'observer à l'aide d'un benchmark, avant d'y répondre par une solution : les huge pages.

1.2 Principe du benchmark de mesure

L'algorithme étudié est implémenté dans le fichier `bench.c`, dans la fonction `benchRead`, à partir de la ligne 111 *(cf. question 1 du sujet)*. Son fonctionnement est le suivant *(cf. question 2, 3 du sujet)* : le buffer est d'abord parcouru une première fois afin de le "préchauffer", c'est-à-dire de garantir que ses données sont déjà présentes dans le cache avant le début de la mesure. Sans cette étape, le chronomètre inclurait le temps de chargement initial depuis la mémoire principale, ce qui fausserait la mesure du temps d'accès purement lié au cache. Une fois cette précondition posée, le chronomètre est démarré et une série de lectures du buffer est effectuée ; le temps mesuré reflète donc uniquement le comportement du système mémoire (cache et TLB) pour une taille de buffer donnée.

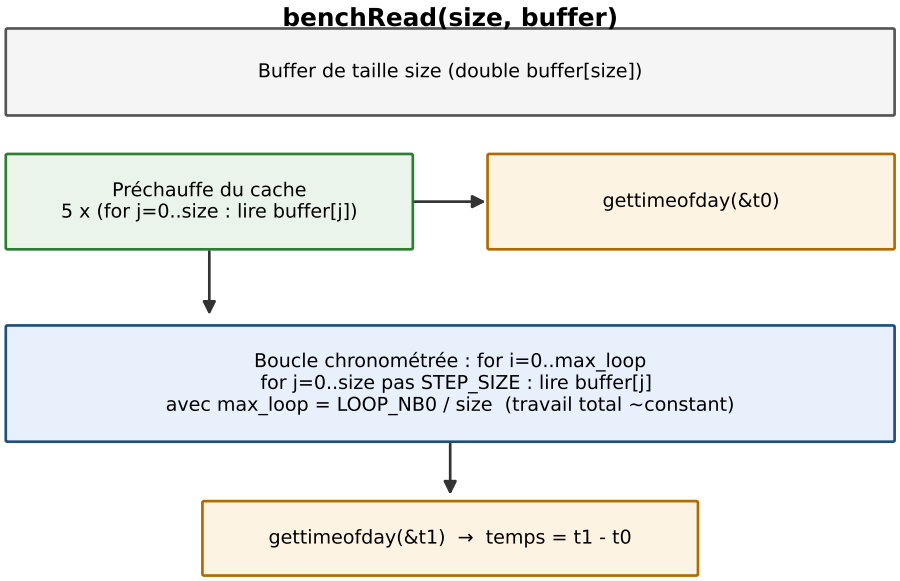


Figure 1: Fonctionnement du benchmark de test des caches

1.3 Résultats avec des pages standards

En faisant varier la taille du buffer et en traçant le temps d'exécution obtenu avec `malloc_bench` *(cf. question 4 du sujet)*, on observe une courbe caractéristique de la hiérarchie mémoire *(cf. question 5 du sujet)* :

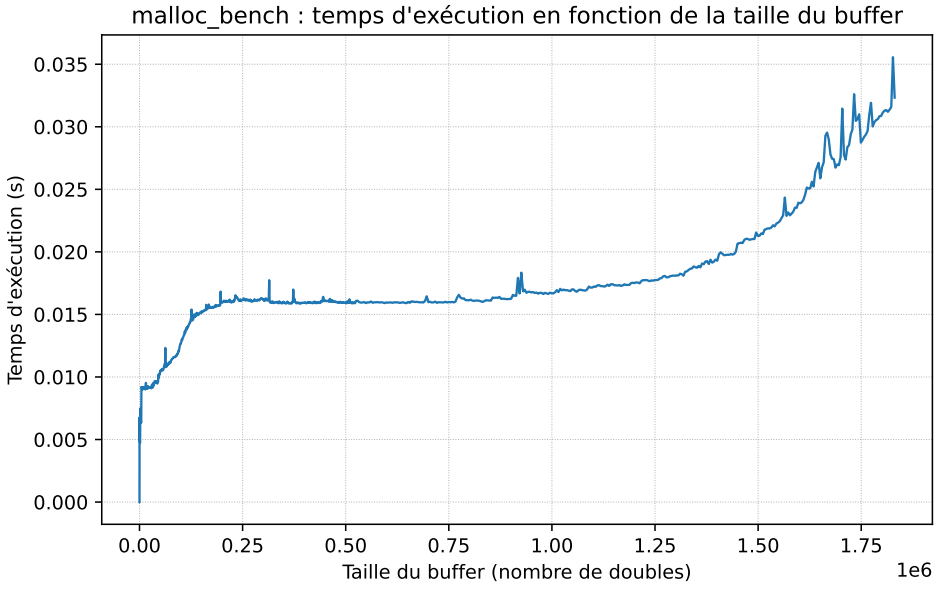


Figure 2: Courbe obtenue avec `malloc_bench` (échelle linéaire)

On distingue une première montée lors du passage du cache L1 au cache L2, puis une seconde lors de la transition du cache L2 au cache L3 (aux alentours de 200 000 doubles). Le temps d'accès reste ensuite relativement stable jusqu'à ce que la capacité du TLB soit dépassée. Au-delà de ce seuil, le temps de lecture croît de façon linéaire : c'est la conséquence directe du phénomène décrit en introduction. Chaque TLB miss oblige le processeur à parcourir les tables de pages en

RAM pour traduire l'adresse virtuelle en adresse physique, puis à mettre à jour le TLB, ce qui introduit un surcoût significatif à chaque accès mémoire. L'utilisation du cache n'est donc pas optimale dès lors que le jeu de données dépasse la couverture du TLB, indépendamment de la taille du cache lui-même.

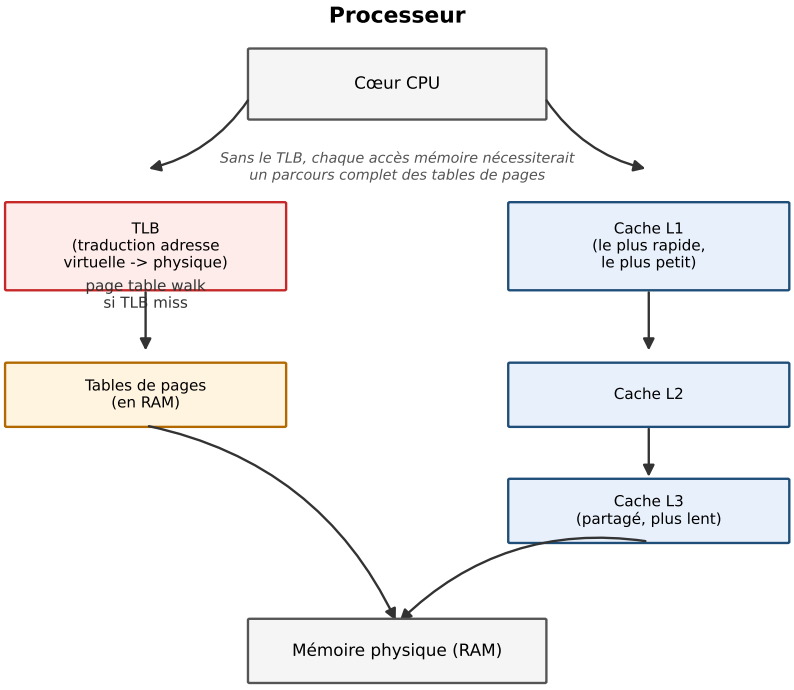


Figure 3: Hiérarchie des caches et rôle du TLB

1.4 Les huge pages comme solution

Une huge page est une page mémoire dont la taille est bien supérieure à celle d'une page standard (4 Ko), typiquement 2 Mo voire 1 Go *(cf. question 6 du sujet)*. L'intérêt est direct : pour couvrir un même volume de données, un nombre bien plus faible de pages est nécessaire, ce qui réduit d'autant le nombre d'entrées consommées dans le TLB.

En activant les huge pages et en relançant la mesure avec `huge_bench` *(cf. question 7 du sujet)*, la courbe obtenue est sensiblement différente :

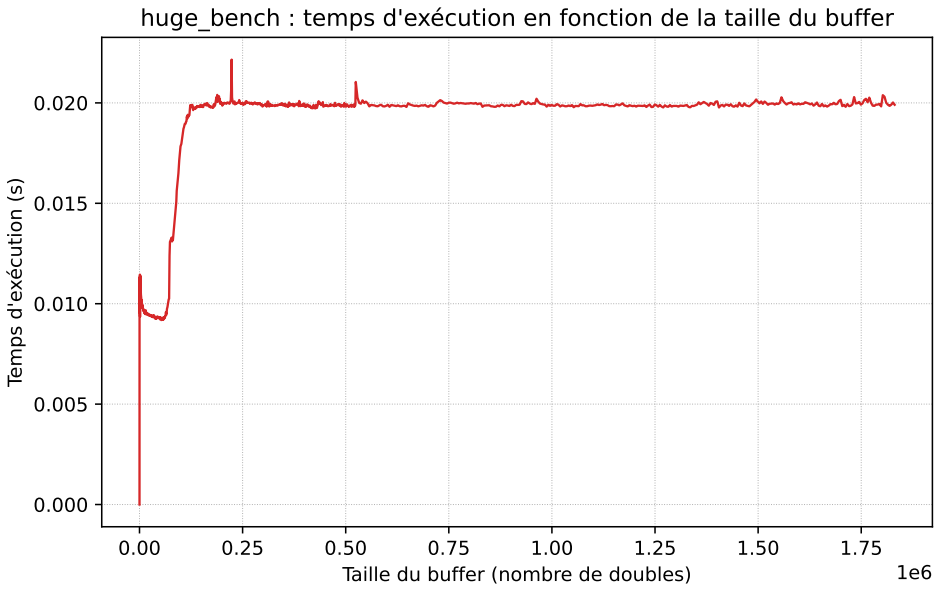


Figure 4: Courbe obtenue avec `huge_bench` (échelle linéaire)

On observe un profil très différent *(cf. question 8 du sujet)*, avec deux points notables. D'une part, la courbe se stabilise sur un unique palier, aux alentours de 0,02 s, dès que le buffer dépasse la capacité du cache L3, et ce palier ne bouge plus jusqu'à la fin de la plage testée : contrairement à `malloc_bench`, aucune remontée continue n'apparaît. D'autre part, et de façon plus surprenante, ce palier se situe *au-dessus* de la courbe de `malloc_bench` sur une bonne partie du graphique : pour des buffers petits à moyens, l'exécution avec huge pages est donc systématiquement plus lente. La comparaison directe des deux courbes permet de bien visualiser ce croisement :

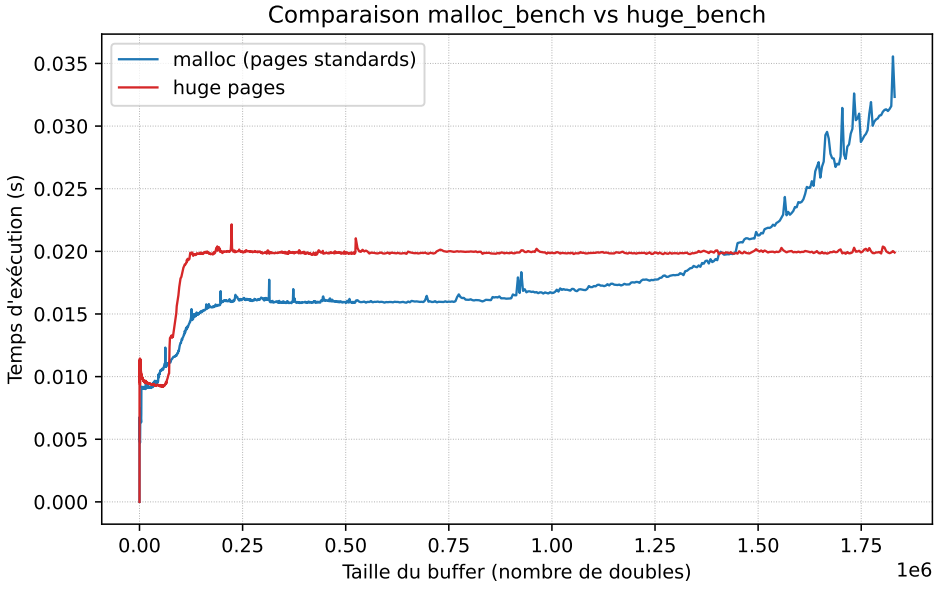


Figure 5: Comparaison des temps d'exécution obtenus avec `malloc_bench` et `huge_bench`

Les deux courbes se croisent aux alentours de 1,4 million de doubles (soit environ 11 Mo, une taille de buffer qui commence à saturer le cache L3). Avant ce point, `huge_bench` affiche un surcoût quasi constant d'environ 2 à 3 ms par rapport à `malloc_bench` : ce surcoût est

vraisemblablement imputable au chemin d'allocation via `hugetlbfs` lui-même (allocation adossée à un fichier plutôt qu'à une simple table de pages classique), indépendant du nombre d'accès effectués, et donc constant quelle que soit la taille du buffer. Après ce point, en revanche, le comportement s'inverse totalement : `malloc_bench` entre dans la zone de TLB misses répétés décrite en Q.5 et son temps d'exécution s'envole, tandis que `huge_bench` reste parfaitement stable sur son palier. Le nombre de pages nécessaires pour couvrir le buffer étant bien plus faible avec des huge pages, la couverture du TLB n'est jamais dépassée sur la plage testée : le facteur limitant redevient la bande passante du cache plutôt que la granularité de la pagination. Dans le cas du processeur utilisé pour ces mesures (Ryzen 7 3800X), c'est cette combinaison — surcoût fixe mais absence totale de TLB misses — qui explique la forme des deux courbes : les huge pages ne sont donc avantageuses que pour des jeux de données suffisamment grands pour que le gain sur les TLB misses compense leur surcoût initial.

Cette solution n'est cependant pas sans contrepartie *(cf. question 9 du sujet)*. D'abord, l'allocation se fait par page entière : toute réservation, même de quelques octets, consomme une page complète de 2 Mo (voire 1 Go), ce qui entraîne une fragmentation interne potentiellement très importante dès lors que de nombreuses petites allocations sont nécessaires. Ensuite, les huge pages doivent être explicitement réservées au niveau du système (via `hugetlbfs` ou `sysctl`), en nombre limité et fixé à l'avance : cette réservation immobilise de la mémoire physique qui ne peut plus être utilisée pour autre chose, même si l'application qui les a demandées n'en a pas encore l'usage. Enfin, les pages ainsi réservées échappent à certains mécanismes de gestion mémoire classiques du noyau, en particulier le swap : une fois allouées, elles ne peuvent généralement pas être swappées, ce qui réduit la flexibilité du système face à la pression mémoire globale.

2 Concevoir un allocateur mémoire contiguë en espace utilisateur

Les huge pages permettent de limiter les TLB misses, mais elles ne garantissent pas à elles seules que la mémoire allouée par un programme soit contiguë en mémoire physique : c'est le rôle du `malloc` standard, combiné à la gestion de la pagination par le noyau, qui échappe au contrôle direct du développeur. La seconde partie du TD propose donc de construire, pas à pas, un allocateur en espace utilisateur (`hp_allocator`) qui réserve une zone de mémoire physique contiguë via les huge pages, puis gère lui-même la distribution de cette zone à travers ses propres `malloc`/`free`.

2.1 Une première implémentation naïve

Le point d'entrée de la bibliothèque repose sur deux fonctions appelées automatiquement au chargement et au déchargement du programme *(cf. question 10 du sujet)* : `hp_init`, appelée avant le lancement du `main` à l'ouverture de la librairie, ouvre le fichier correspondant à la librairie (désigné par la variable `FILE_NAME`) via les flux noyau, puis réserve une certaine quantité de mémoire ; `hp_finalize`, appelée après le `main`, se charge de fermer la librairie.

Pour distribuer cette zone réservée, l'allocateur doit se souvenir, d'un appel à l'autre, de la quantité de mémoire déjà distribuée : c'est le rôle de la variable `alloue` ("allouée") *(cf. question 11 du sujet)*. Elle est déclarée `static`, ce qui la rend persistante et commune à tous les appels de la fonction, plutôt que réinitialisée à chaque appel comme le serait une variable locale classique. Elle est ainsi stockée dans le segment de données du programme, et non sur la pile ni sur le tas.

La première version de `__hp_malloc` correspond à l'implémentation la plus simple envisageable *(cf. question 12 du sujet)* : elle vérifie qu'il reste suffisamment de place dans la zone réservée et, si c'est le cas, y ajoute un nouveau bloc à la suite de ceux déjà distribués.

```c
static void *__hp_malloc(size_t taille)
{
    /*
     * Alloue un bloc de memoire au moins de taille 'taille'.
     */

    static size_t alloue = 0UL;
    void *ret;

    assert(taille > 0);
    // La fonction calcule_multiple_align() determine le plus petit
    // multiple de l'alignement maximal egal ou superieur a a.
    // Celle-ci nous permet d'"arrondir" la taille demandee de
    // sorte que les blocs alloues soit toujours correctement alignes.

    taille = calcule_multiple_align(taille);

    if (taille < MEM_SIZE - alloue)
    {
        fprintf(stderr, "Il n'y a plus assez de memoire disponible.\n");
        return NULL;
    }

    ret = (void*) (alloue+1);
    alloue += taille;
    return ret;
}
```

(cf. question 13 du sujet)

Cette approche fonctionne, mais elle souffre d'un défaut structurel *(cf. question 14 du sujet)* : la mémoire n'est jamais réutilisée. Chaque nouvelle allocation avance uniquement le curseur `alloue` vers la fin de la zone réservée, sans jamais tenir compte des blocs éventuellement libérés entre-temps. Une fois la fin de la zone atteinte, plus aucune allocation n'est possible, même si de la mémoire a été libérée entre-temps et pourrait être réutilisée.

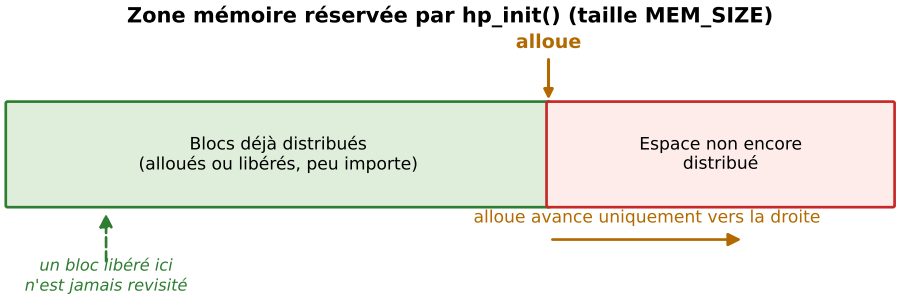


Figure 6: Limite de l'allocateur naïf : la mémoire libérée n'est jamais réutilisée

2.2 Introduction d'une liste de blocs libres

Pour corriger cette limitation, l'allocateur doit garder la trace des blocs libérés afin de pouvoir les réutiliser : c'est le rôle de la liste chaînée `libre`, alimentée par `__hp_free` et consultée par `__hp_malloc` avant toute extension de la zone réservée.

La fonction `recherche_bloc_libre` parcourt cette liste chaînée jusqu'à trouver un bloc dont

la taille est suffisante, ou jusqu'à atteindre la fin de la liste *(cf. question 15 du sujet)*. Dès qu'un bloc suffisamment grand est trouvé, il est retiré de la chaîne (en reliant le bloc précédent directement au bloc suivant, ou en avançant la tête de liste `libre` s'il s'agit du premier élément) puis renvoyé à l'appelant ; si aucun bloc ne convient, la fonction renvoie `NULL`.

```
static struct bloc *recherche_bloc_libre(size_t taille)
{
    /*
     * Recherche un bloc libre ou une suite de blocs libres dont la
         taille
     * est au moins egale a `taille`.
     */

    struct bloc *bloc = libre;
    struct bloc *precedent = NULL;
    struct bloc *ret = NULL;

    while (bloc != NULL)
    {
        if (bloc->taille >= taille)
        {
            if (precedent != NULL)
                precedent->suivant = bloc->suivant;
            else
                libre = bloc->suivant;

            ret = bloc;
            break;
        }

        precedent = bloc;
        bloc = bloc->suivant;
    }

    return ret;
}
```

(cf. question 16 du sujet)

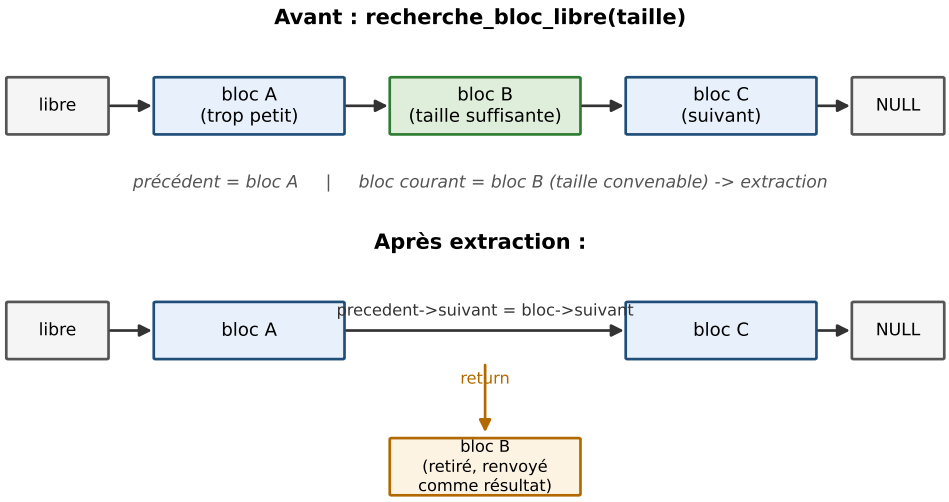


Figure 7: Recherche et extraction d'un bloc libre dans la liste chaînée

`__hp_malloc` est alors modifiée pour exploiter cette liste *(cf. question 17 du sujet)* : on recherche d'abord un bloc libre de taille suffisante parmi ceux déjà libérés ; ce n'est que si aucun bloc ne convient qu'un nouveau bloc est créé en repoussant le curseur `alloue`, à condition que la mémoire restante soit suffisante.

```
static void *__hp_malloc(size_t taille)
{
    /*
     * Alloue un bloc de memoire au moins de taille `taille`.
     */

    static size_t alloue = 0UL;
    void *ret;

    assert(taille > 0);

    // La fonction calcule_multiple_align() determine le plus petit
    // multiple de l'alignement maximal egal ou superieur a a.
    // Celle-ci nous permet d'"arrondir" la taille demandee de
    // sorte que les blocs alloues soit toujours correctement alignes.
    taille = calcule_multiple_align(taille);

    ret = recherche_bloc_libre(taille); // recherche d'un bloc libre

    if (!ret)
    {
        if (MEM_SIZE - alloue < TAILLE_EN_TETE + taille)
        {
            fprintf(stderr, "Il n'y a plus assez de memoire disponible.\
               n");
            return NULL;
        }

        ret = reserve.mem + alloue + TAILLE_EN_TETE;
        alloue += taille;
        bloc_init(ret, taille);
    }

    return ret;
}
```

(cf. question 18 du sujet)

Symétriquement, `__hp_free` doit permettre de reverser un bloc dans cette liste *(cf. question 19 du sujet)* : après avoir vérifié que le pointeur fourni n'est pas `NULL`, la fonction parcourt la chaîne des blocs déjà libérés jusqu'à son extrémité, puis y ajoute le nouveau bloc libéré.

```c
static void __hp_free(void *ptr)
{
    /*
     * Ajoute le bloc fourni a la liste des blocs libres.
     */

    struct bloc *bloc = libre;
    struct bloc *nouveau;

    if (!ptr)
        return;

    nouveau = (struct bloc *)((char *)ptr - TAILLE_EN_TETE);
    nouveau->suivant = NULL;

    while (bloc && bloc->suivant != NULL)
        bloc = bloc->suivant;

    if (bloc == NULL)
        libre = nouveau;
    else
        bloc->suivant = nouveau;
}
```

(cf. question 20 du sujet)

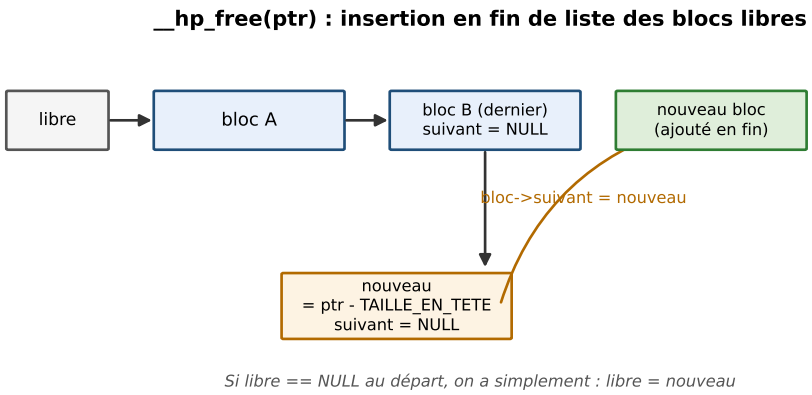


Figure 8: Insertion d'un bloc libéré dans la liste des blocs libres

Avec cette liste de blocs libres, l'allocateur corrige le défaut identifié plus haut : la mémoire libérée redevient disponible pour de futures allocations, sans attendre que la zone réservée soit épuisée. On notera toutefois que cette implémentation reste simple : elle ne fusionne pas les blocs libres adjacents (pas de *coalescing*) et sélectionne le premier bloc de taille suffisante plutôt que le plus ajusté, ce qui peut entraîner de la fragmentation à long terme.

2.3 Mise à disposition sous forme de bibliothèque

La transformation de l'allocateur en bibliothèque dynamique (*(cf. question 21 du sujet)*), destinée à permettre son utilisation par d'autres programmes sans recompilation, n'a pas pu être menée à bien dans le temps imparti pour ce TD. En conséquence, les mesures demandées en lien avec cette bibliothèque (*(cf. question 22 du sujet)*, comparaison avec `svalat_bench_lib`) ainsi que le wrapping de `malloc`/`free` qui s'appuie dessus (*(cf. question 23 du sujet)*) n'ont pas pu être réalisés.

Conclusion

Ce TD illustre comment la pagination, mécanisme invisible du point de vue applicatif, peut avoir un impact mesurable sur les performances via son interaction avec les caches et le TLB. Les huge pages apportent une réponse à ce problème en réduisant la pression sur le TLB au prix d'un surcoût fixe et d'une fragmentation interne accrue, un compromis qui ne devient avantageux qu'à partir d'une certaine taille de jeu de données. Leur intégration dans un allocateur dédié permettrait, une fois la bibliothèque dynamique et le wrapping de `malloc`/`free` menés à terme, d'en faire bénéficier n'importe quel programme de façon transparente.